

# Solar Live WiFi Manual

solar\_ws0129-main

2026-06-17

## 1 目的

**live\_wifi** モードは、同一 WiFi 上の Raspberry Pi / マイコンから UTF-8 JSON を UDP で送受信しつつ、本番ライブ運用を 1 本で回すためのモードである。

- ソーラーカー状態受信
- 伴走車状態受信
- 予報 API 自動取得
- 風予報のオンライン補正
- 上位プランナ 1 時間更新
- 下位プランナ 1 Hz 速度司令
- ソーラーカー側と伴走車側への司令再送
- ダッシュボード表示
- ログ保存

起動入口は次である。

```
.\SolarSim.ps1 -Mode live_wifi -Action up
```

## 2 構成

```
Solar car Raspberry Pi ----\  
                                > UDP JSON -> telemetry_text_bridge_node -> ROS topics  
Chase car Raspberry Pi ----/  
  
Open-Meteo API -> weather_fetch_node -> raw forecast CSV  
raw forecast CSV + route + live observation -> wind_correction_node -> corrected forecast CSV  
corrected forecast CSV -> mpc_node -> planner commands  
planner commands -> telemetry_text_bridge_node -> UDP JSON -> solar / chase  
planner topics -> dashboard_node -> http://localhost:8080  
planner and telemetry -> logger_node -> outputs/logs/*.csv
```

主 launch は `launch/solar_race_live_wifi.launch.py` である。

### 3 風速予測補正

添付資料 風速予測補正方法.pdf の考え方は `mpc_solarcar/wind_correction_node.py` に実装している。

#### 3.1 観測値

- 最優先観測は `headwind_ms`
- それが無い場合は `wind_speed_ms`, `wind_dir_deg`, `course_deg` から進行方向成分へ変換
- `course_deg` も無い場合は route waypoint から方位を補間

成分化は次で行う。

$$\text{headwind} = \text{wind\_speed\_ms} \cdot \cos(\text{wind\_from\_deg} - \text{heading\_deg})$$

#### 3.2 現在時刻の補正

予報平均  $\mu_{\text{fcst}}$  と観測  $y_{\text{obs}}$  を、予報分散  $\sigma_{\text{fcst}}^2$  と観測分散  $\sigma_{\text{obs}}^2$  で融合する。

$$\mu_{\text{post}} = \frac{\mu_{\text{fcst}}/\sigma_{\text{fcst}}^2 + y_{\text{obs}}/\sigma_{\text{obs}}^2}{1/\sigma_{\text{fcst}}^2 + 1/\sigma_{\text{obs}}^2}$$

$$\sigma_{\text{post}}^2 = \frac{1}{1/\sigma_{\text{fcst}}^2 + 1/\sigma_{\text{obs}}^2}$$

#### 3.3 将来時刻への伝播

現在の補正量  $\Delta = \mu_{\text{post}} - \mu_{\text{fcst}}$  を、距離減衰付きで将来へ伝播する。

$$w(s) = \exp(-ds_{\text{km}}/L_c)$$

$$\mu_{\text{corr}} = \mu_{\text{fcst, future}} + w\Delta$$

`use_exp_distance_decay=false` の場合は時間減衰へ切り替える。

#### 3.4 不確かさの増加

CSV に標準偏差列が無いときは次で予報標準偏差を作る。

$$\sigma_{\text{fcst}}^2(t) = \sigma_0^2 + \alpha \cdot \text{lead\_h}$$

ここで  $\sigma_0$  は `forecast_sigma0_ms`、 $\alpha$  は `forecast_variance_growth_per_hour` である。

### 3.5 MPC に渡す値

planning\_quantile を使って headwind\_ms を決める。

- 0.5: 中央値
- 0.84: 約  $+1\sigma$
- 0.975: 約  $+2\sigma$

## 4 YAML 項目

編集入口は config/solar/bwsc\_2027\_demo.yaml である。

### 4.1 必須に近い項目

- paths.route\_waypoints\_csv
- paths.route\_profile\_csv
- live.weather.raw\_forecast\_csv
- live.wind\_model.corrected\_forecast\_csv
- live.wifi\_bridge.bind\_host
- live.wifi\_bridge.bind\_port
- live.wifi\_bridge.solar\_remote\_host
- live.wifi\_bridge.solar\_remote\_port
- live.wifi\_bridge.chase\_remote\_host
- live.wifi\_bridge.chase\_remote\_port

### 4.2 風補正パラメータ

| YAML                                                | 意味            |
|-----------------------------------------------------|---------------|
| live.wind_model.measurement_sigma_ms                | 風観測ノイズ        |
| live . wind _ model . correlation _ distance_km     | 観測補正が効く距離     |
| live . wind _ model . fallback _ correlation_time_h | 時間減衰時の時定数     |
| live.wind_model.forecast_sigma0_ms                  | 予報初期標準偏差      |
| live.wind_model.forecast_variance_growth_per_hour   | 先読み時間ごとの分散増加  |
| live.wind_model.planning_quantile                   | MPC に入れる風の分位点 |
| live.wind_model.confidence_z                        | 表示用信頼区間係数     |
| live.wind_model.min_sigma_ms                        | 標準偏差の下限       |

| YAML                                                | 意味                                |
|-----------------------------------------------------|-----------------------------------|
| <code>live.wind_model.preferred_source</code>       | <code>auto, vehicle, chase</code> |
| <code>live.wind_model.use_exp_distance_decay</code> | 距離減衰を使うか                          |

### 4.3 WiFi ブリッジ

| YAML                                             | 意味          |
|--------------------------------------------------|-------------|
| <code>live.wifi_bridge.enabled</code>            | 全体有効化       |
| <code>live.wifi_bridge.enable_inbound</code>     | 受信有効化       |
| <code>live.wifi_bridge.enable_outbound</code>    | 送信有効化       |
| <code>live.wifi_bridge.bind_host</code>          | 受信 bind     |
| <code>live.wifi_bridge.bind_port</code>          | 受信 port     |
| <code>live.wifi_bridge.publish_period_sec</code> | 送信周期        |
| <code>live.wifi_bridge.send_to_solar</code>      | ソーラーカー送信有効化 |
| <code>live.wifi_bridge.send_to_chase</code>      | 伴走車送信有効化    |

## 5 推奨ネットワーク設定

| 機器                  | 推奨 IP                      | 用途            |
|---------------------|----------------------------|---------------|
| プランナ PC             | <code>192.168.50.10</code> | 受信サーバ、ダッシュボード |
| ソーラーカー Raspberry Pi | <code>192.168.50.21</code> | 車両送信、司令受信     |
| 伴走車 Raspberry Pi    | <code>192.168.50.22</code> | 伴走車送信、司令受信    |

| Port               | 向き                                   | 用途          |
|--------------------|--------------------------------------|-------------|
| <code>52001</code> | <code>vehicle/chase → planner</code> | 状態送信        |
| <code>52002</code> | <code>planner → solar</code>         | 速度司令受信      |
| <code>52003</code> | <code>planner → chase</code>         | 伴走車表示・記録用受信 |

送受信ルールは次とする。

- 1 datagram = 1 JSON object
- UTF-8
- 改行不要
- 基本周期 1 Hz
- 単位は km/h, m/s, deg, km, V, A, degC

## 6 Planner が受ける JSON

### 6.1 vehicle\_state

```
{
  "type": "vehicle_state",
  "ts_unix": 1781700000.0,
  "speed_kmh": 72.4,
  "soc": 0.83,
  "batt_temp_c": 34.1,
  "batt_current_a": 8.6,
  "batt_voltage_v": 97.8,
  "s_kmh": 412.6,
  "lat": -19.13542,
  "lon": 146.81235,
  "alt_m": 18.4,
  "wind_speed_ms": 6.8,
  "wind_dir_deg": 118.0,
  "course_deg": 176.5
}
```

風は headwind\_ms を直接送るか、wind\_speed\_ms, wind\_dir\_deg, course\_deg を送る。

### 6.2 chase\_state

```
{
  "type": "chase_state",
  "ts_unix": 1781700000.0,
  "speed_kmh": 74.0,
  "lat": -19.13580,
  "lon": 146.81210,
  "alt_m": 18.1,
  "wind_speed_ms": 6.5,
  "wind_dir_deg": 121.0,
  "course_deg": 176.0
}
```

## 7 Planner から送る JSON

```
{
  "type": "planner_command",
  "schema": "solar_v1",
  "ts_unix": 1781700000.0,
  "planner": {
    "speed_cmd_kmh": 71.0,
    "upper_speed_cmd_kmh": 73.5,
    "drive_mode": "eco"
  },
  "summary": {
    "race_progress_pct": 13.6,
    "next_stop_dist_kmh": 58.2,
    "next_stop_eta_min": 47.0,
    "finish_dist_kmh": 2622.9,
    "finish_eta_h": 41.5,
    "avg_plan_speed_kmh": 72.1
  },
  "wind": {
    "plan_headwind_ms": 4.8,
    "mean_headwind_ms": 4.2,
    "std_headwind_ms": 1.1,
    "lo95_headwind_ms": 2.0,
    "hi95_headwind_ms": 6.4
  }
}
```

マイコン側の最低限の使い方は次である。

- `planner.speed_cmd_kmh`: 実速度司令
- `planner.upper_speed_cmd_kmh`: 参考表示
- `planner.drive_mode`: ローカル map 切替
- `summary.*`: ナビ表示
- `wind.*`: 状況表示と記録

watchdog は 3 秒を推奨する。

## 8 送信側と受信側の参考実装

- `scripts/wifi_vehicle_sender_example.py`
- `scripts/wifi_chase_sender_example.py`
- `scripts/wifi_planner_receiver_example.py`
- `templates/network/vehicle_sender.service.example`
- `templates/network/chase_sender.service.example`
- `templates/network/esp32_planner_receiver_example.ino`

動作確認例:

```
python scripts/wifi_vehicle_sender_example.py --host 192.168.50.10 --port 52001
python scripts/wifi_chase_sender_example.py --host 192.168.50.10 --port 52001
python scripts/wifi_planner_receiver_example.py --bind_port 52002
```

## 9 ダッシュボード

ダッシュボードは 1 画面の数値ボックス主体へ変更している。主表示は次である。

- 指令速度、上位速度、実測速度
- SOC、電池温度、電圧、電流、電力
- 太陽電力、勾配
- 風の観測、予報、補正平均、標準偏差、95% 区間、計画値
- 進捗率、次 stop、finish 距離、ETA
- vehicle / chase GPS
- forecast status, wind model status, network status

## 10 起動手順

1. config/solar/bwsc\_2027\_demo.yaml の IP / port / route / weather path を合わせる
2. Raspberry Pi 側送信プログラムを配置する
3. .\SolarSim.ps1-Modelive\_wifi-Actionup を実行する
4. 必要なら .\SolarSim.ps1-Modelive\_wifi-Actiongraph を実行する
5. 終了時は .\SolarSim.ps1-Modelive\_wifi-Actionstop

## 11 確認項目

- networkstatus が更新される
- windmodelstatus に source=vehicle または source=chase が出る
- outputs/runtime/live\_forecast\_raw.csv が更新される
- outputs/runtime/live\_forecast\_corrected.csv が更新される
- outputs/logs/solar\_live\_\*.csv が増える
- rqt\_graph\_solar\_live\_wifi.png が保存できる